```
        if (msg.sender != chairperson) {revert();}                    ❺
        if (x < state) revert();                                      ❻
        state = x;
    }
    }
```

❶ Internally coded as 0,1,2,3

❷ Constructor makes contract deployer the chairperson

❸ Number of proposals is a parameter for the constructor

❹ State changer function

❺ Only chairperson can change state; otherwise, revert

❻ State has to progress in 0,1,2,3 order; otherwise, revert

In listing 3.2, only the constructor for BallotV2 and the `changeState()` functions are shown. When you copy this code into the Remix IDE, be sure to preface it with the contents of listing 3.1. The complete BallotV2.sol is available in the codebase of this chapter.

Let's review the code. Initially, the `state` variable is set to `Init` by static initialization. The account (`msg.sender`) invoking the constructor is designated as the chairperson. More appropriately, we could say that the chairperson initiates the ballot process by deploying the smart contract. The number of proposals is initialized, and the chairperson's vote is given a weight of 2 (arbitrarily). Voting-phase changes are affected by the function `changeState()`; you can enforce that only the chairperson can change from one phase to another and that `Phase` can take only the values `{0,1,2,3}` for `{Init, Regs, Vote, Done}`. You want the phases to progress from `Init` to `Done` via `Regs` and `Vote`. Let's explore how all this works:

1. Enter the code for the BallotV2 smart contract into the Remix IDE, and check the state change functionality.